



泊位分配问题：从数学建模到lua仿真

——物流信息管理——

姓名：栾智健

学号：2220232872

目标

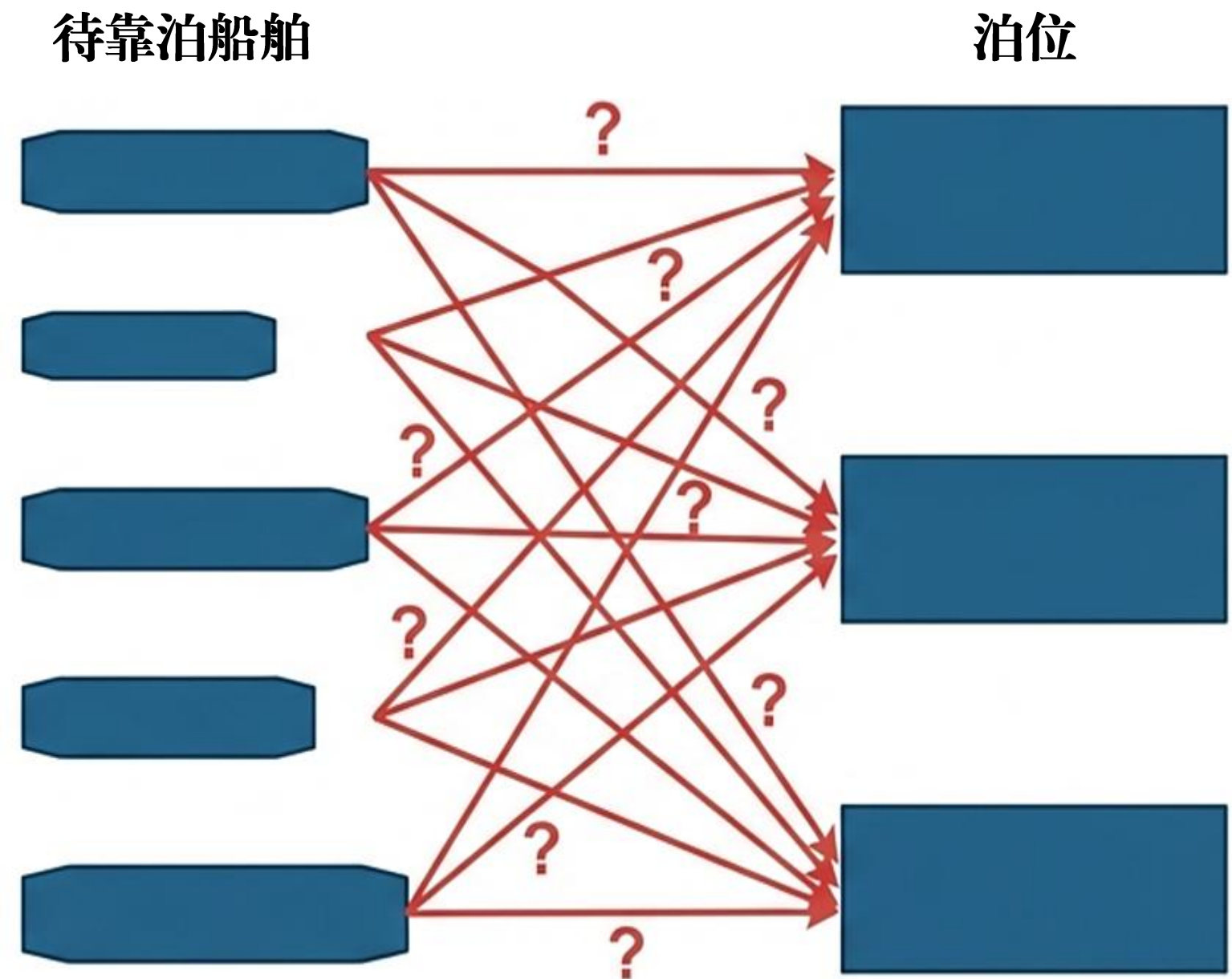
将一组待靠泊船舶分配到有限的泊位，以最小化总运营成本。

挑战

资源受限以及船舶与泊位水位的各异性导致分配困难

权衡

不同的分配方案会导致不同的处理时间、转运成本和等待时间



参数	解释
V	待靠泊船舶集合, $j, i \in V$
B	泊位集合, $b \in B$
t_{1jb}	船舶 j 在泊位 b 的装卸作业时间
c_{1jb}	船舶 j 在泊位 b 的装卸成本
t_{2jb}	船舶 j 的货物从泊位 b 到堆场的转运时间
c_{2jb}	船舶 j 的货物从泊位 b 到堆场的转运成本
r_b	泊位 b 的释放时间
c_3	单位时间成本
c_{4b}	泊位 b 的固定使用成本
A_j	船舶 j 的预计到港时间
L_b	泊位 b 的长度
D_b	泊位 b 的水深
l_j	船舶 j 的船长
d_j	船舶 j 的吃水
M	一个足够大的正数

参数	解释
y_{jb}	0-1 变量, 若船舶 j 停靠在泊位 b , 则为 1, 否则为 0
x_{ij}	0-1 变量, 若在同一泊位上, 船舶 i 先于 船舶 j 作业, 则为 1, 否则为 0
z_j	连续变量, 船舶 j 的作业完成时间
α_b	0-1 变量, 若泊位 b 被使用, 则为 1, 否则为 0。

$$\min Z = \underbrace{\sum_{b \in B} \sum_{j \in V} (c_{1jb} + c_{2jb}) y_{jb}}_{\text{作业与转运成本}} + \underbrace{c_3 \sum_{j \in V} \left(z_j + \sum_{b \in B} t_{2jb} y_{jb} \right)}_{\text{时间成本}} + \underbrace{\sum_{b \in B} c_{4b} \alpha_b}_{\text{固定成本}}$$

最小化总成本

物理空间约束

唯一性

$\sum y_{jb} = 1$ (Each ship gets 1 berth).

船长限制

$$l_j \cdot y_{jb} \leq L_b$$

吃水限制

$$d_j \cdot y_{jb} \leq D_b$$



时间与逻辑约束

到港限制

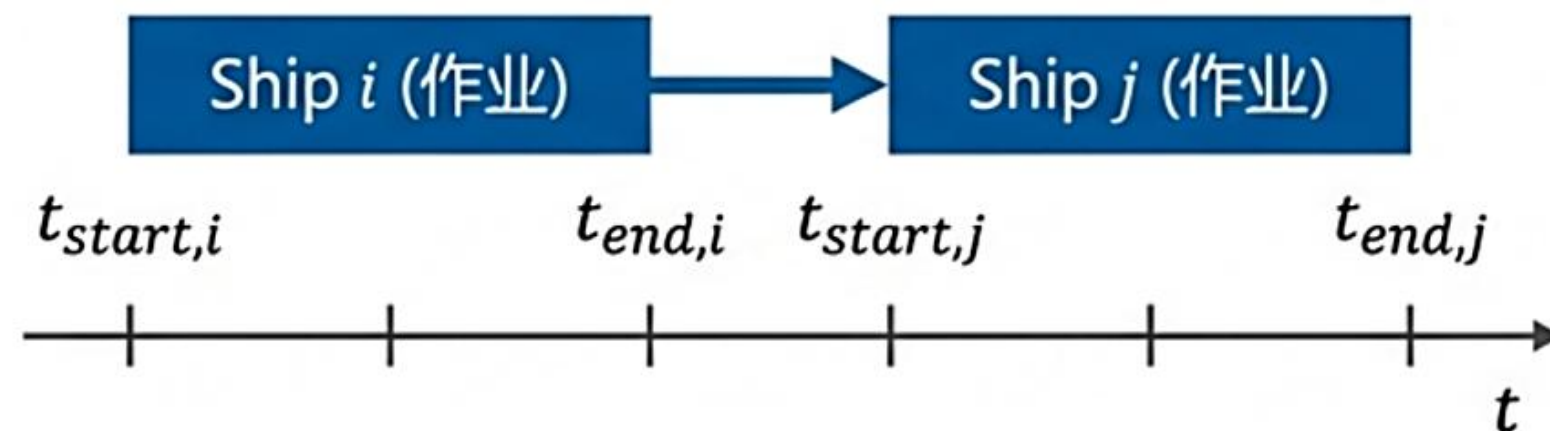
开始时间 $\geq A_j$

时序逻辑

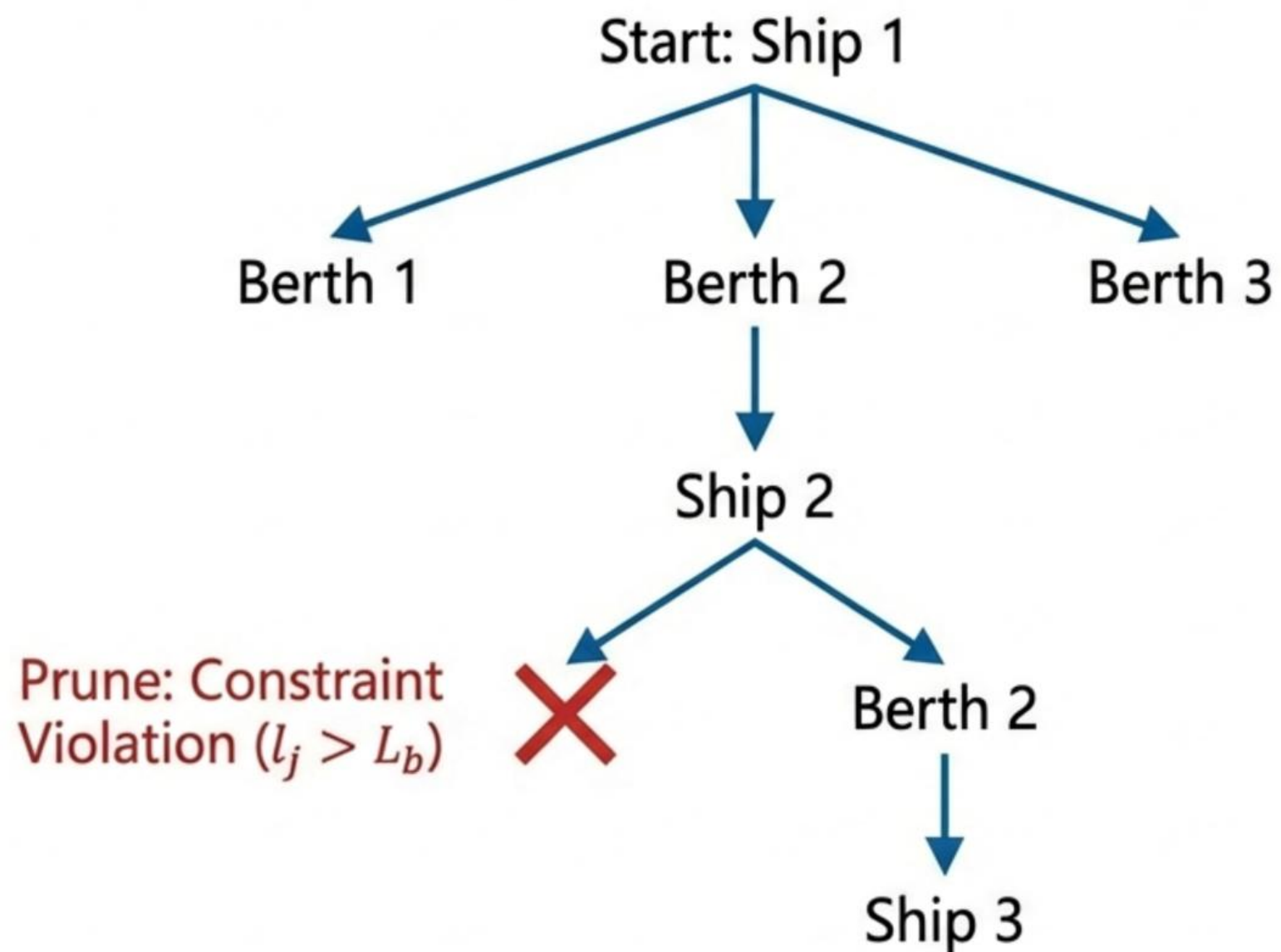
使用大M法防止时间重叠 (Eq 2, 3)

泊位可用

作业不能早于 r_b



回溯算法



算法流程

1. 遍历船舶
2. 尝试分配泊位
3. 剪枝：检查物理约束
4. 递归
5. 回溯
6. 更新全局最优

输入数据

使用lua随机生成仿真场景

```
1 math.randomseed(os.time()) -- 使用当前时间设置随机数种子，确保每次运行26
2 local Params = { -- 定义一个名为Params的表，用于存储数学模型所需的参数27
3     V = {}, -- 初始化待靠泊船舶集合V，对应数学模型参数表中的 V 28
4     B = {}, -- 初始化泊位集合B，对应数学模型参数表中的 B 28
5     M = 99999, -- 定义大M参数，用于逻辑约束中的线性化处理，对应参数29
6     c3 = 50, -- 定义单位时间成本系数，对应参数表中的 c3 30
7     numV = 5, -- 设定待分配的船舶总数量为5艘 31
8     numB = 3 -- 设定港口的泊位总数量为3个 32
9 } -- 参数表定义结束 33
10
11 local GlobalOpt = { -- 定义一个全局变量表，用于存储回溯算法找到的全34
12     minZ = math.huge, -- 初始化最小目标函数值Z为无穷大，以便后续比较35
13     assignment = {} -- 初始化一个空表，用于存储对应的最佳分配方案 36
14 } -- 全局最优解结构定义结束 37
15
16 function GenerateData() -- 定义数据生成函数，用于随机创建泊位和船舶38
17     for b = 1, Params.numB do -- 开始循环，遍历每一个泊位，从1到numB39
18         table.insert(Params.B, { -- 向泊位集合Params.B中插入一个新的40
19             id = b, -- 设定泊位ID，对应下标 b 40
20             L = math.random(150, 300), -- 随机生成泊位长度 L_b，范围141
21             D = math.random(12, 20), -- 随机生成泊位水深 D_b，范围142
22             r = 0, -- 设定泊位释放时间 r_b，初始假设为0 43
23             c4 = math.random(100, 300) -- 随机生成泊位固定使用成本 44
24         }) -- 泊位数据插入结束 44
25     end -- 泊位生成循环结束 45
26
27     for j = 1, Params.numV do -- 开始循环，遍历每一艘船，从1到numV
28         local vessel = { -- 定义一个局部变量vessel，存储当前船舶的属性
29             id = j, -- 设定船舶ID，对应下标 j
30             A = math.random(0, 10), -- 随机生成船舶预计到港时间 A_j，范围0到10
31             l = math.random(80, 150), -- 随机生成船长 l_j，范围80到150米
32             d = math.random(8, 15), -- 随机生成船舶吃水 d_j，范围8到15米
33             params = {} -- 初始化该船舶针对不同泊位的作业参数表
34         } -- 船舶基础属性定义结束
35         for b = 1, Params.numB do -- 嵌套循环，遍历每一个泊位，生成船舶j在泊位b的特定参数
36             vessel.params[b] = { -- 在params表中记录针对泊位b的参数
37                 t1 = math.random(5, 10), -- 随机生成装卸时间 t_1jb，范围5到10
38                 c1 = math.random(100, 500), -- 随机生成装卸成本 c_1jb，范围100到500
39                 t2 = math.random(1, 3), -- 随机生成转运时间 t_2jb，范围1到3
40                 c2 = math.random(50, 100) -- 随机生成转运成本 c_2jb，范围50到100
41             } -- 参数赋值结束
42         end -- 嵌套循环结束
43         table.insert(Params.V, vessel) -- 将生成的船舶数据插入到船舶集合Params.V中
44     end -- 船舶生成循环结束
45 end -- 数据生成函数结束
```

确保每一次的仿真场景具备随机性和鲁棒性

核心求解函数：约束映射

约束 (7) & (8):

$$l_j \leq L_b \text{ AND } d_j \leq D_b$$



```
local v = Params.V[vesselIndex]
for bIdx, b in ipairs(Params.B) do
    -- 映射数学约束
    local checkLength = (v.l <= b.L)
    local checkDraft = (v.d <= b.D)

    if checkLength and checkDraft then
        -- 递归求解
        BacktrackSolve(vesselIndex + 1, ...)
    end
end
end
```

复现约束

```
local v = Params.V[vesselIndex] -- 获取当前需要分配的船舶对象 v
for bIdx, b in ipairs(Params.B) do -- 遍历每一个可能的泊位 b
    local checkLength = (v.l <= b.L) -- 检查约束条件(7): 船长 l_j 是否小于等于泊位长度 L_b
    local checkDraft = (v.d <= b.D) -- 检查约束条件(8): 吃水 d_j 是否小于等于泊位水深 D_b
```

对所有约束完美复现

$$l_j \cdot y_{jb} \leq L_b, \forall j \in V, \forall b \in B \quad (7)$$

船长限制：船长不能超过泊位长度。

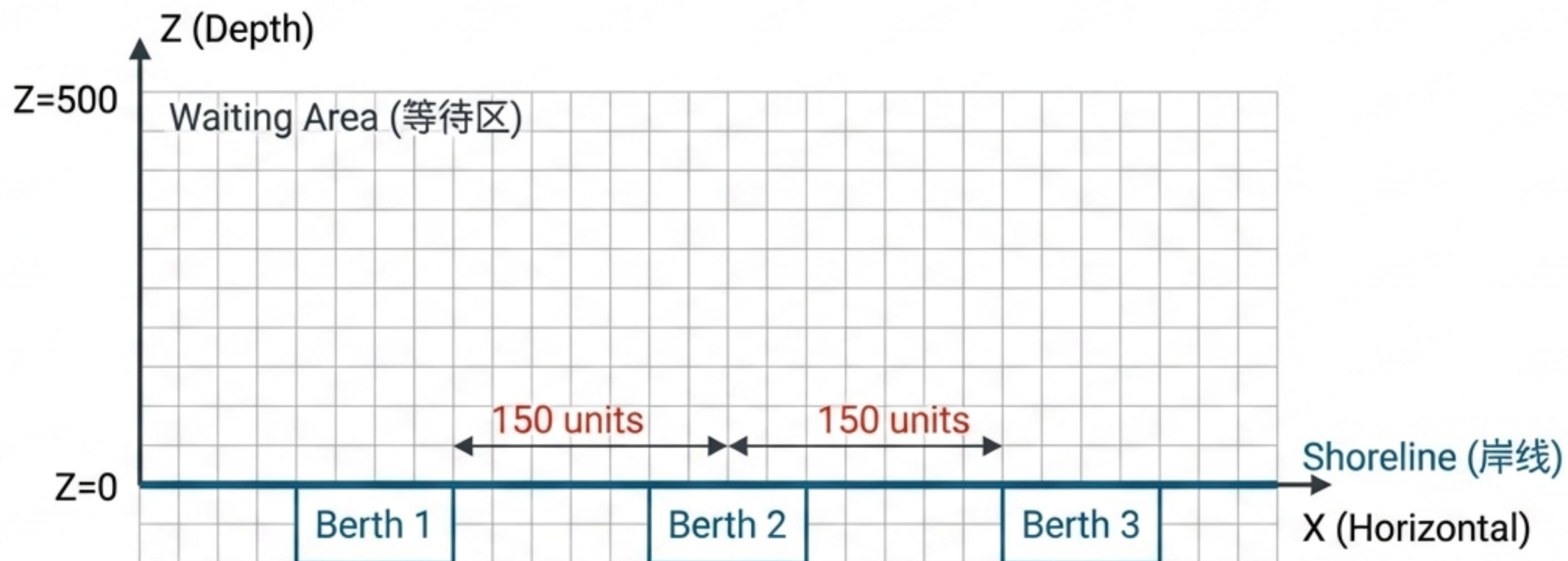
$$d_j \cdot y_{jb} \leq D_b, \forall j \in V, \forall b \in B \quad (8)$$

吃水限制：船舶吃水不能超过泊位水深。

```
[2026/1/16 14:37:14]>===== 模型参数数据 =====  
[2026/1/16 14:37:14]>--- 泊位信息 (B) ---  
[2026/1/16 14:37:14]>ID Length(L) Depth(D) FixCost(c4)  
[2026/1/16 14:37:14]>1 292 20 134  
[2026/1/16 14:37:14]>2 278 14 191  
[2026/1/16 14:37:14]>3 213 15 207  
[2026/1/16 14:37:14]>--- 船舶信息 (V) ---  
[2026/1/16 14:37:14]>ID Arrival(A) Length(l) Draft(d)  
[2026/1/16 14:37:14]>1 1 122 13  
[2026/1/16 14:37:14]>2 8 148 11  
[2026/1/16 14:37:14]>3 9 137 13  
[2026/1/16 14:37:14]>4 1 113 11  
[2026/1/16 14:37:14]>5 6 94 9  
[2026/1/16 14:37:14]>-----  
[2026/1/16 14:37:14]>---开始求解---  
[2026/1/16 14:37:14]>===== 最终优化结果 =====  
[2026/1/16 14:37:14]>最小目标函数值 Z: 5740.00  
[2026/1/16 14:37:14]>详细调度方案:  
[2026/1/16 14:37:14]>Vessel(j) Berth(b) Start Finish(z_j)  
[2026/1/16 14:37:14]>1 3 1 8  
[2026/1/16 14:37:14]>4 2 1 7  
[2026/1/16 14:37:14]>5 2 7 12  
[2026/1/16 14:37:14]>2 1 8 14  
[2026/1/16 14:37:14]>3 3 9 19
```

根据此结果进行仿真
每次运行结果具备随机性

三维场景搭建



-- 岸桥模型加载

```
local rmqc = scene.addobj('/res/ct/rmqc.glb')  
rmqc:setpos(bx, 0, -30)
```

-- 船舶模型初始化 (等待区)

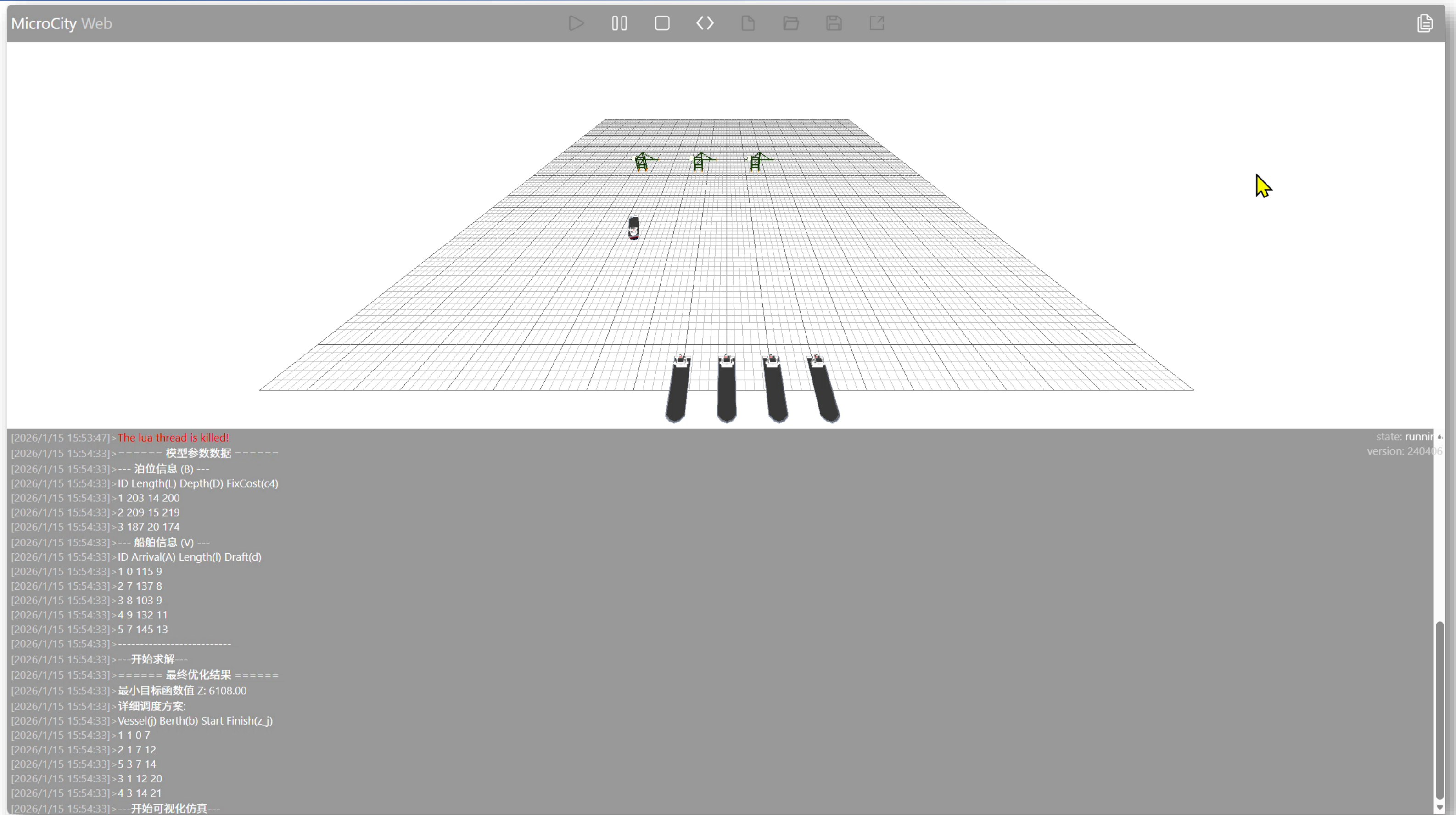
```
ship:setpos(waitX, 0, 500)
```

```
local berthViz = {} -- 初始化可视化泊位表，用于存储泊位坐标
local berthWidth = 150 -- 设定泊位之间的间距为150，防止模型拥挤
local startX = -((Params.numB * berthWidth)/2) -- 计算第一个泊位的X坐标，使所有泊位整体居中

for i, bData in ipairs(Params.B) do -- 遍历生成的泊位数据，创建3D模型
    local bx = startX + (i-1) * berthWidth -- 计算当前泊位的X轴坐标
    local rmqc = scene.addobj('/res/ct/rmqc.glb') -- 在场景中添加岸桥模型代表泊位
    if rmqc then -- 如果模型加载成功
        rmqc:setpos(bx, 0, -30) -- 设置岸桥位置，Z轴设为-30使其位于岸边
        rmqc:setrot(0, math.pi, 0) -- 旋转岸桥180度（pi弧度），使其朝向海面
    end -- 模型设置结束
    berthViz[i] = { x = bx, z = 40 } -- 保存该泊位的停靠点坐标（Z=40为海面作业点）
end -- 泊位创建循环结束

local shipViz = {} -- 初始化可视化船舶表，用于控制动画
for idx, vData in ipairs(Params.V) do -- 遍历生成的船舶数据，创建3D模型
    local ship = scene.addobj('/res/ct/ship.glb') -- 在场景中添加集装箱船模型
    local waitX = (idx * 50) - 150 -- 计算等待区的X坐标，将船排开避免重叠
    if ship then -- 如果模型加载成功
        ship:setpos(waitX, 0, 500) -- 设置初始位置，Z=500代表远海等待区
        ship:setscale(0.8, 0.8, 0.8) -- 缩小模型比例至0.8，避免视觉过大
    end -- 模型设置结束
    shipViz[idx] = { -- 在shipViz表中保存船舶对象和状态
        obj = ship, -- 保存3D对象引用
        data = vData, -- 保存船舶数据引用
        state = "waiting", -- 初始化状态为“等待中”
        task = nil -- 初始化任务为空，稍后绑定
    } -- 结构保存结束
end -- 船舶创建循环结束
```

仿真展示





感谢观看



——物流信息管理——

姓名：栾智健

学号：2220232872